

漏洞分析报告

漏洞名称	【通过路径遍历上传 Web shell】
测试平台	PortSwigger Web Security Academy
漏洞类型	【文件上传漏洞】
风险等级	高危
测试时间	2026 年

一、漏洞原理

攻击者在上传文件时使用 URL 编码混淆绕过检查，欺骗服务器把文件保存到本该不允许存放脚本的目录之外，从而让这个 PHP 文件能被 Web 服务器直接访问并执行。

二、漏洞危害

攻击者可借此读取服务器任意敏感文件（如 `/home/carlos/secret`）、执行系统命令、反弹 Shell，进而完全控制 Web 服务器，并以此为跳板攻击内网其他主机，造成数据泄露、服务瘫痪乃至整个系统沦陷。

三、复现过程

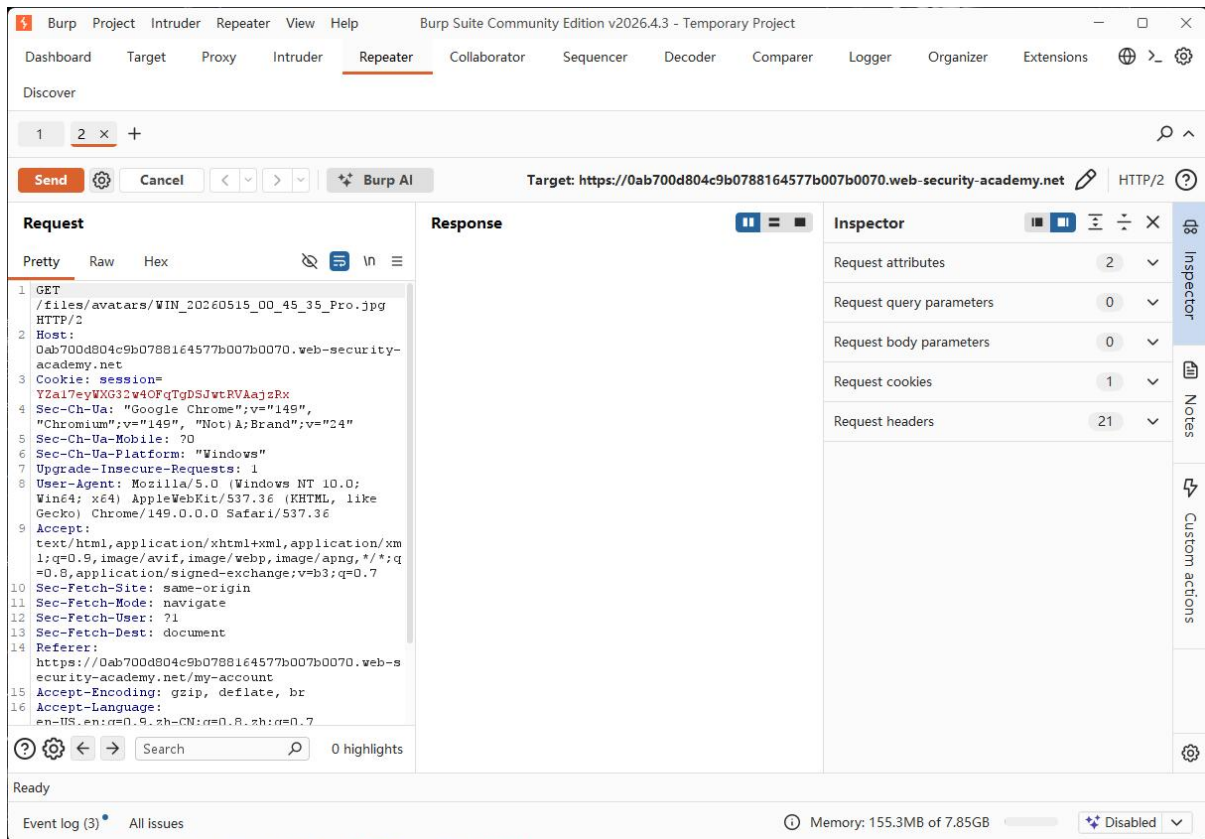
3.1 测试环境

测试平台：PortSwigger Web Security Academy 靶场

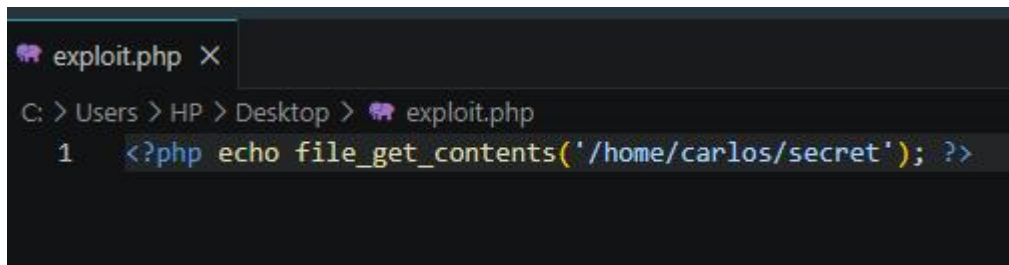
使用工具：Burp Suite、浏览器

3.2 测试步骤

Step1: 上传头像并用 burp suite 抓包



Step2: 在本地创建 exploit.php 并尝试作为头像上传

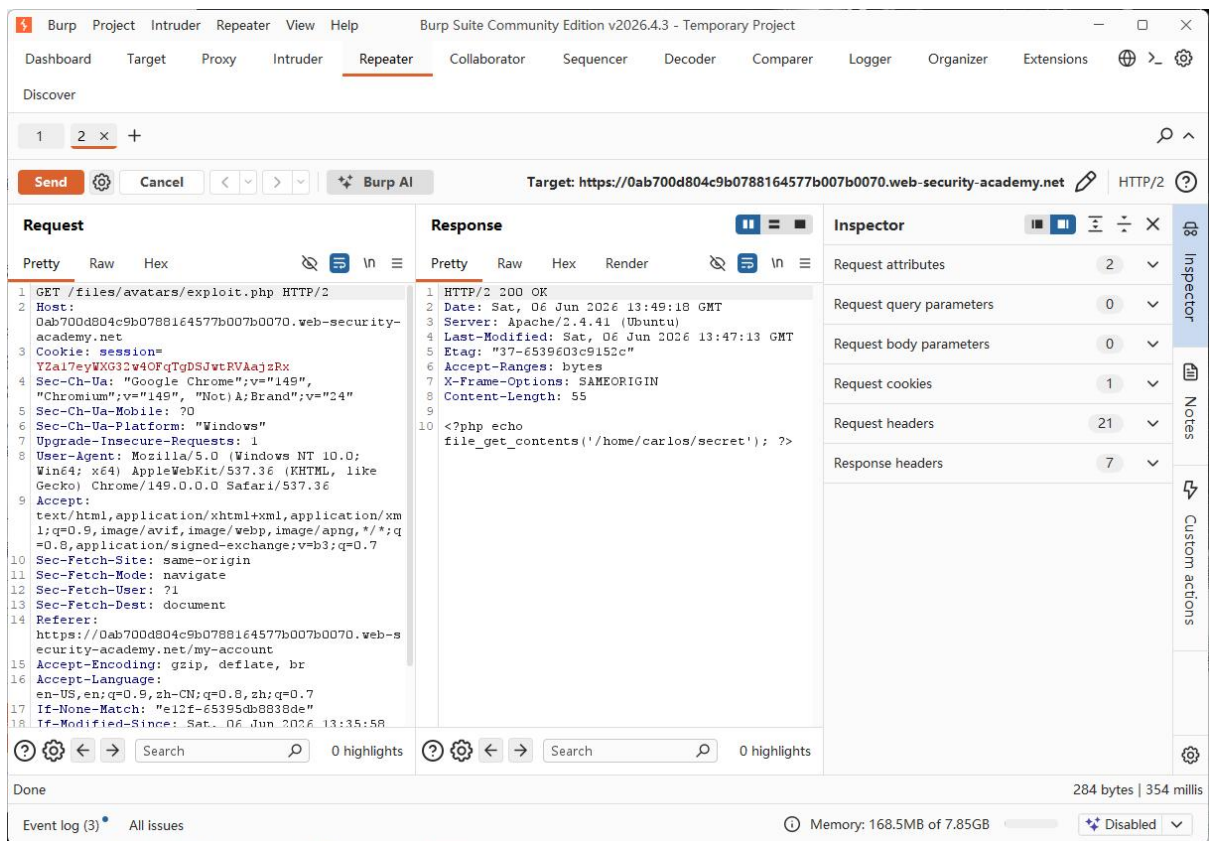


可以看到返回上传成功

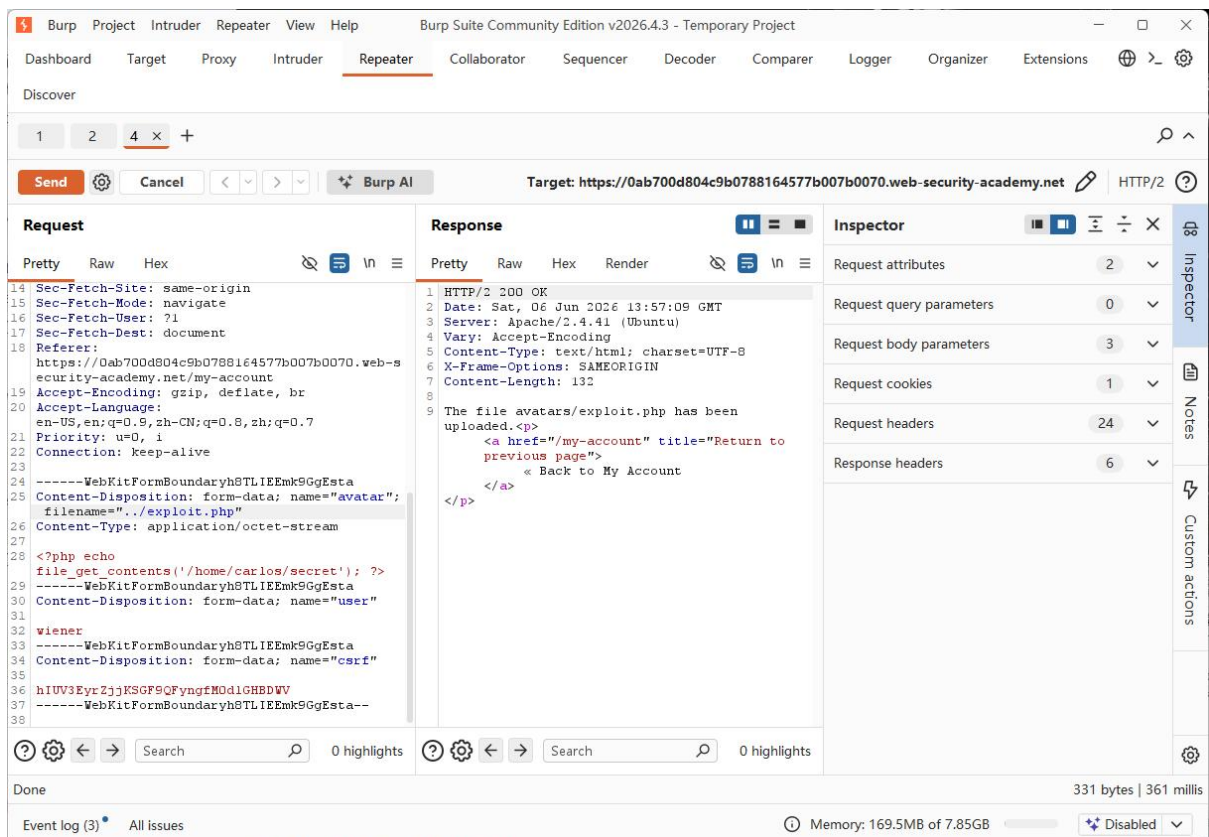
The file avatars/exploit.php has been uploaded.

[Back to My Account](#)

Step3: 尝试发送 GET 请求执行脚本，服务器没有执行脚本并返回输出，而是直接返回 php 文件的内容作为纯文本

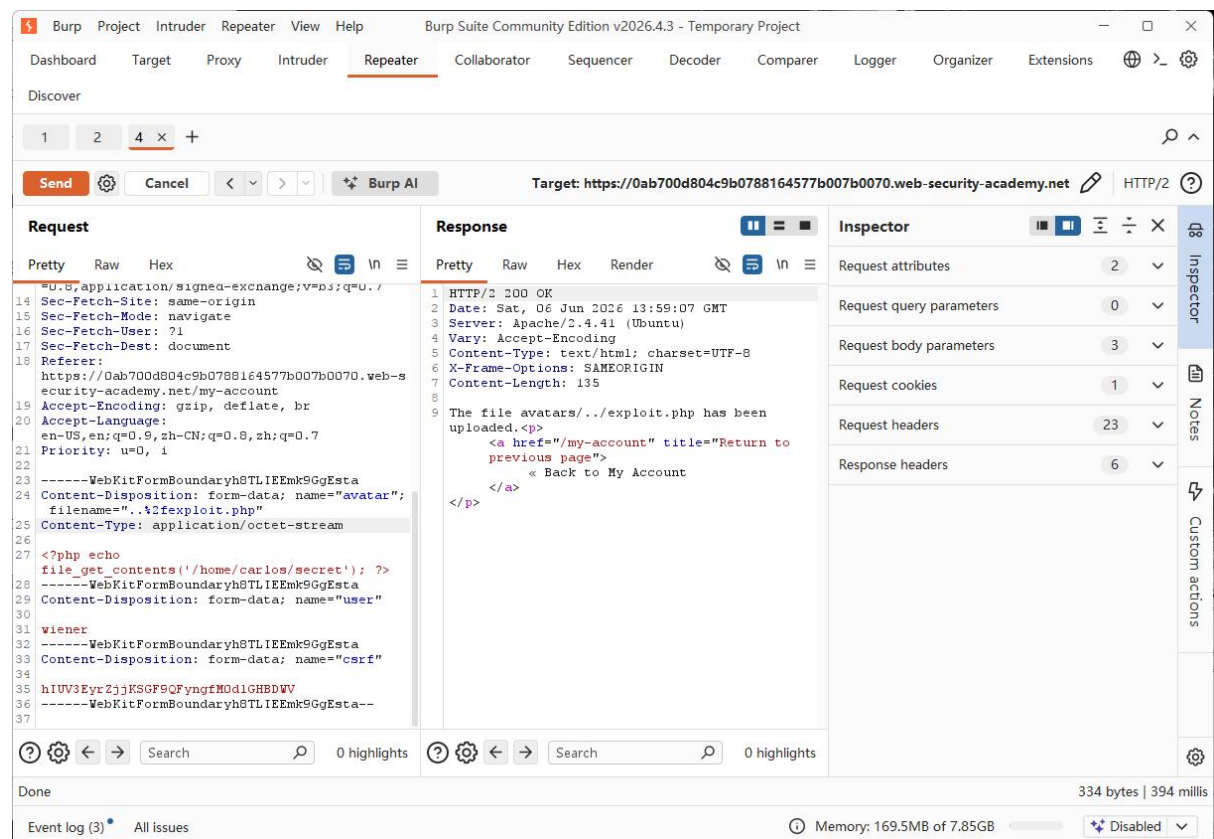


尝试修改 Content-Disposition 当中 filename，利用路径遍历将 php 脚本写到可执行的目录



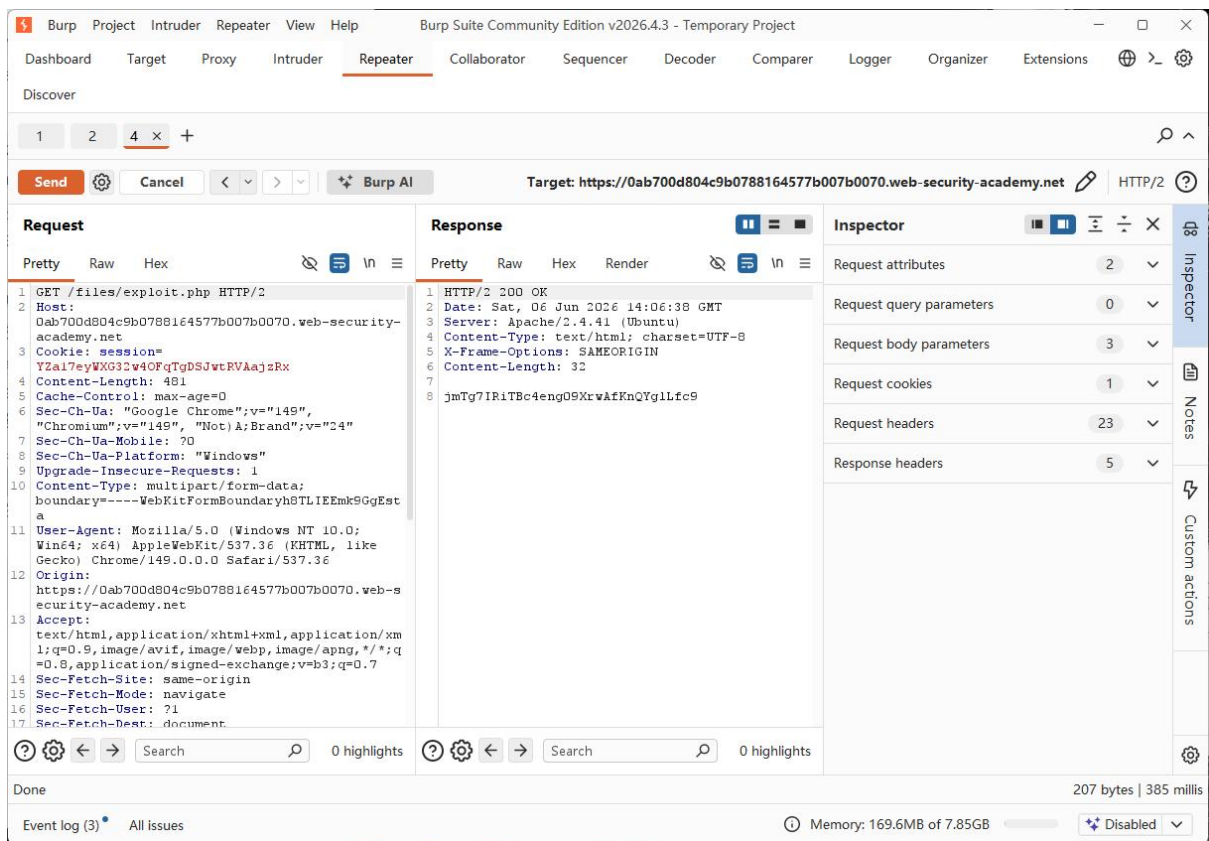
通过回显得出服务器剥离了目录遍历序列

尝试通过 URL 编码混淆



返回 The file avatars/../../exploit.php has been uploaded.表示服务器对文件名进行了 URL 解码

构造 GET 请求执行脚本，返回了 secret



四、注意事项

测试时需注意：服务器虽会检查并拦截 `../`，但可使用 URL 编码 `%2f` 绕过，即 `filename="..%2fexploit.php"`。若仍失败，可尝试多级回退（如 `..%2f..%2fexploit.php` 后访问 `/exploit.php`），并确保 `Content-Type` 保持 `image/jpeg` 以通过前端校验。

五、修复建议

修复方案	说明
丢弃用户提供的原始文件名，使用服务器生成的随机字符串	攻击者利用 <code>../</code> 或编码变种 <code>%2f</code> 的目的是改变文件保存路径，跳出预期的上传目录。通过彻底重命名，可从根本上阻断路径遍历攻击，同时避免因编码绕过导致的过滤失效
配置 Web 服务器（如 Apache、Nginx），将所有用户上传的文件保存到 Web 根目录（如 <code>/var/www/html/</code> ）以外的独立文件夹（如 <code>/data/uploads/</code> ），并通过一个独立的、不做脚本解析的静态资源服	即使攻击者成功上传了 <code>shell.php</code> ，只要该文件不在 Web 可执行目录下，就无法通过 HTTP 直接触发脚本解析。将上传目录与执行环境物理隔离，是防御 <code>webshell</code> 执行的最有效手段之一。

务来提供访问（或使用 X-Accel-Redirect 等内部转发机制）	
使用白名单方式：仅允许特定类型的文件（如 image/jpeg、image/png），并利用可靠库（如 PHP 的 finfo、Java 的 Tika）读取文件内容中的魔术字节（magic bytes）进行二次验证。对于图片，还可以使用 GD 库重新编码以去除潜在的恶意代码。	攻击者常伪造 Content-Type: image/jpeg 但实际内容为 PHP 代码。仅检查 HTTP 头或文件扩展名很容易被绕过。通过内容签名校验，即使文件名被改成 ../exploit.php，实际内容不是合法图片也会被拒绝上传。

六、参考资料

- PortSwigger Web Security Academy: <https://portswigger.net/web-security>
- OWASP Top 10: <https://owasp.org/Top10/>